C语言——面向过程编程

单元三、项目重构

第三次课 变量相关

一、回顾变量

1 类型关键字
2 命名规范
3 字面值后缀
4 字节大小与数据范围
5 溢出现象
6 进制转换
7 内存存储原理：原码 补码 反码
8 左值 右值
9 八进制 十六进制字面值
10 打印格式

二、enum 枚举类型

基本概念：当某些要表达的信息其信息种类是有限个数时，可以用枚举类型增强代码的可读性。

例如：性别仅有男 或女两种可能的值，此种情况就可以用到枚举。再比如血型、一年四季、星座、时辰、一周七天。

枚举类型的定义：

定义枚举格式(一)：　**enum 枚举类型名 ={枚举值1，枚举值2 ,....};**

```
#include<stdio.h>
/*定义枚举类型*/
enum 血型 {A,B,AB,O} ;
int main()
{

     //声明枚举类型的变量
    enum 血型 xx;
    xx=A;
    printf("%d \n",xx);
    xx=B;
    printf("%d \n",xx);
    xx=AB;
    printf("%d \n",xx);
    xx=O;
    printf("%d \n",xx);
    return 0;
}
```

```
enum 血型 {A,B,AB,O} ;
int main()
{

    //声明枚举类型的变量
    enum 血型 xx;
    xx=A;
    printf("%d \n",xx);
    xx=B;
    printf("%d \n",xx);
    xx=AB;
    printf("%d \n",xx);
    xx=O;
    printf("%d \n",xx);
    return 0;
}
```

```
C:\WINDOWS\system32\cmd.exe
0
1
2
3
请按任意键继续. . .
```

根据打印结果可知： 枚举类型的枚举值默认从0开始 依次编号。

也可以用循环遍历枚举的每个成员值：

```
#include<stdio.h>
/*定义枚举类型*/
enum 血型 {A,B,AB,O} ;
int main()
{

    //声明枚举类型的变量
    enum 血型 xx;
    for(xx=A;xx<=O;xx++)
    {
        printf("%d\n",xx);
    }
    return 0;
}
```

定义枚举格式 (二)：

enum 枚举类型名 ={枚举值1=某整数，枚举值2 =某整数,....};

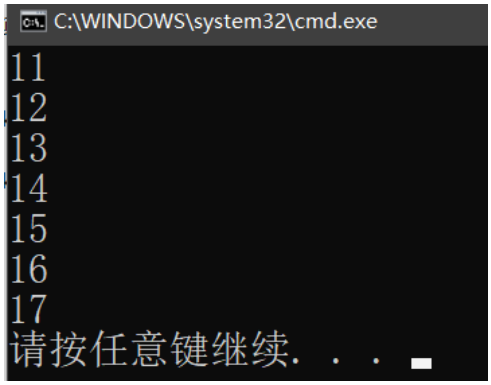
```
#include<stdio.h>
/*定义枚举类型*/
enum FANGXIANG
{
    up=1,down=3,left=2,right=0
} fx=up;//在定义之后 直接声明枚举类型的遍历并赋初始值。
int main()
{
    switch (fx)
    {
    case up:
        break;
    case down:
        break;
    case left:
        break;
    case right:
        break;
```

```
    }
    return 0;
}
```

定义枚举格式 (三):

enum 枚举类型名 ={枚举值1 =起始值, 枚举值2 ,....};

```
#include<stdio.h>
/*定义枚举类型*/
enum WEEK
{
    MON=11,TUE,WED,THU,FRI,SAT,SUN
} week;
int main()
{
    for( week=MON;week<=SUN;week++)
    {
        printf("%d\n",week);
    }

    return 0;
}
```



根据结果可以看到：第一个赋起始值后，后面就在起始值基础上累加。

定义枚举格式 (四):

typedef enum{ 枚举值1,枚举值2,枚举值3.... } 枚举类型名;

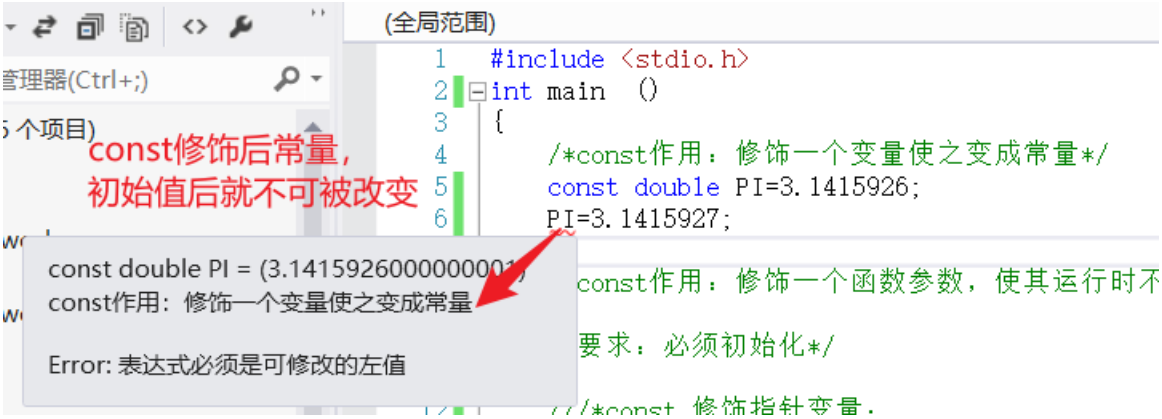
```
#include<stdio.h>
/*定义枚举类型*/
typedef enum
{
    MON ,TUE,WED,THU,FRI,SAT,SUN
} WEEK;
int main01()
{
     WEEK w = MON;//用了typedef起名后 ，在定义变量时就不需要写enum关键字了。

    return 0;
}
```

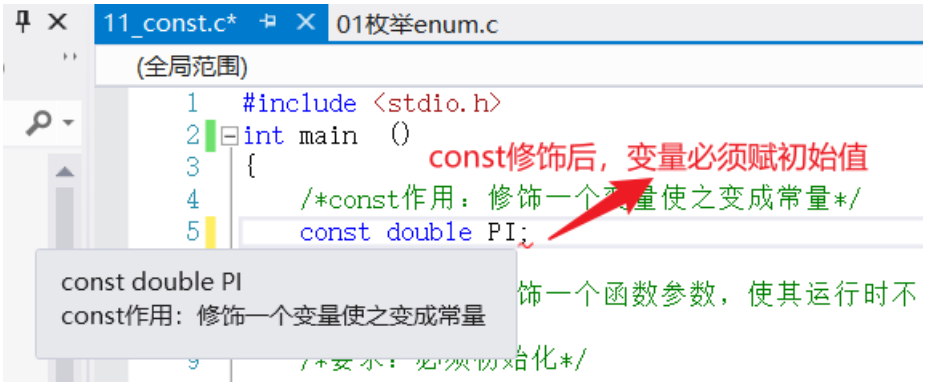
总之：枚举的存在就是增强的代码的可读性

三、const 常量修饰符

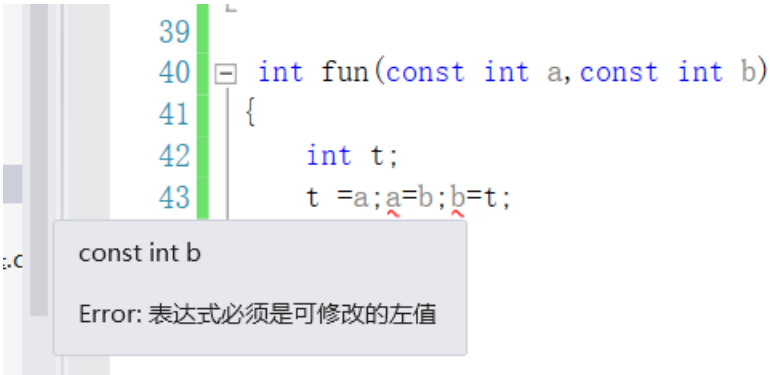
const 是 constant 的缩写，意思是“恒定不变的”！它是定义只读变量的关键字，或者说 const 是定义常变量的关键字。

① 修饰一个变量使之变成常量





② 修饰一个函数形式参数，使其运行时不可更改



③const 修饰指针变量

【明确：】指针通常包括两个方面的内容：A：指针保存的地址 B：指针指向的空间数据

因此const修饰指针也从两个方面进行修饰：

```
1. 指针变量不可指向新地址
2. 指针变量指向的地址空间内的数据不可修改
```

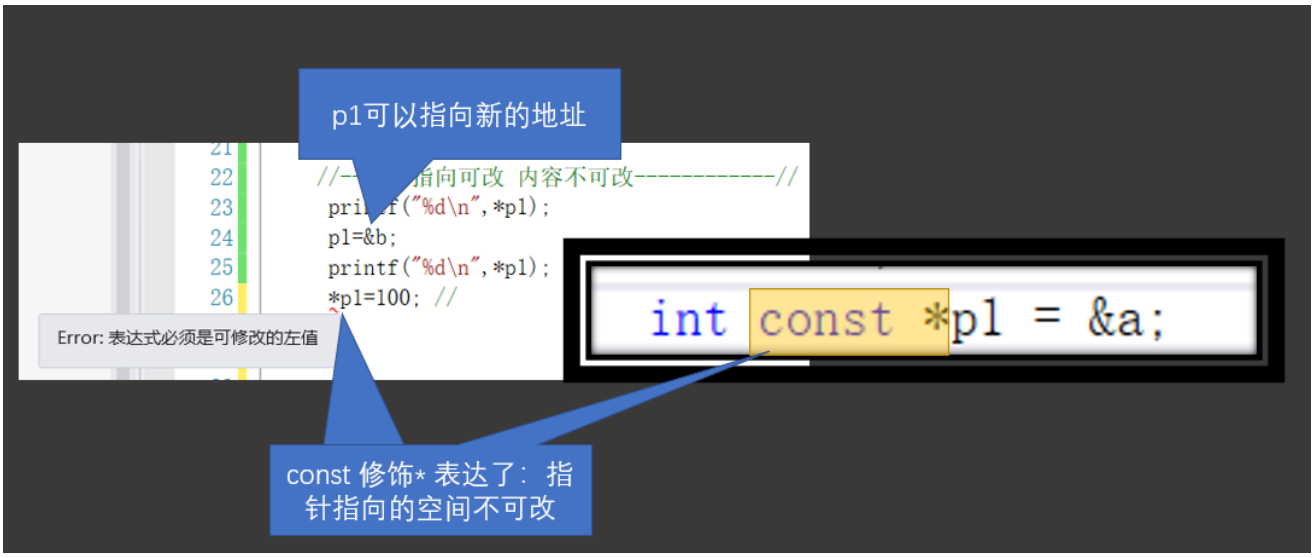
测试案例：

定义相关变量：

```c
int a=10;
int b=20;
//注意下面：const出现的位置
int const *p1 = &a;
int * const p2=&a;
int const * const p3 = &a;
```

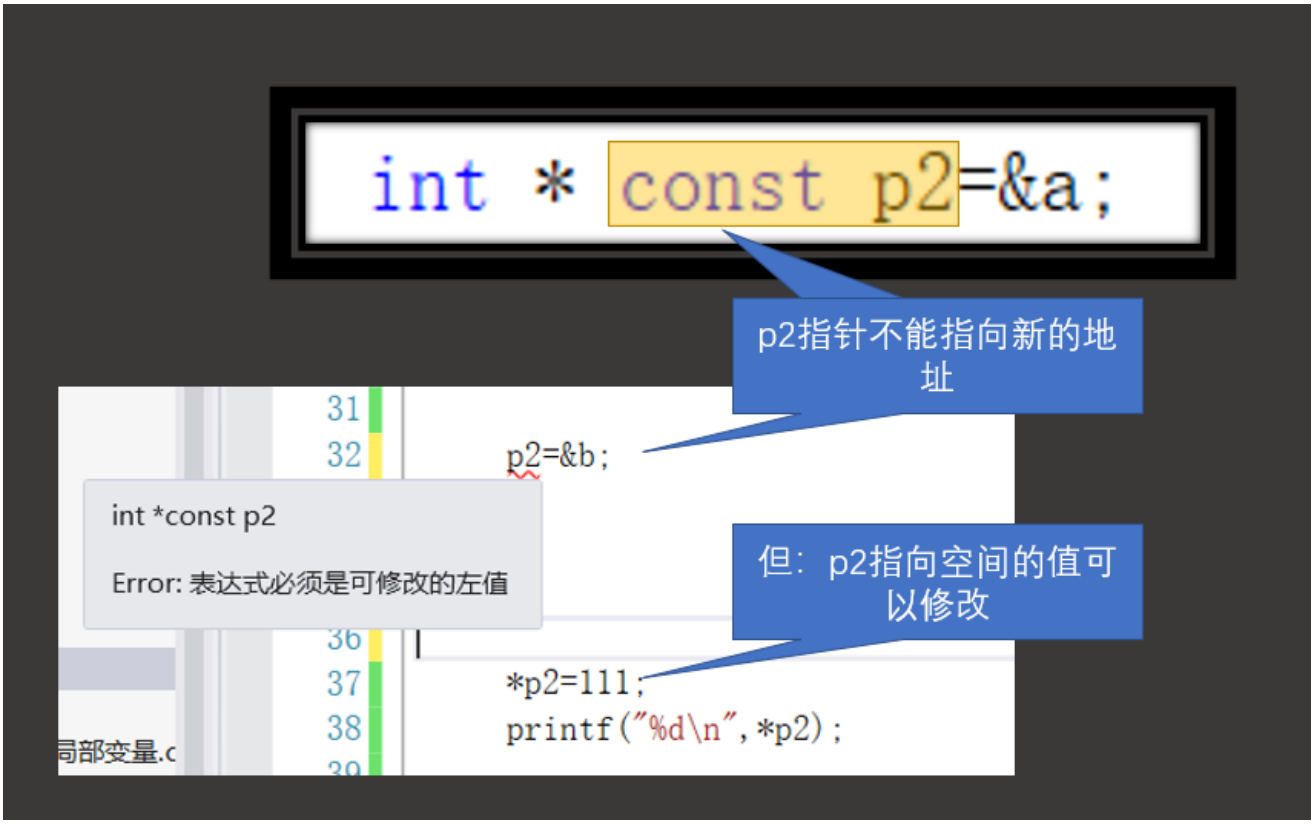
A：const 修饰* —— 指向可改 内容不可改

```c
    printf("%d\n",*p1);
    p1=&b;
    printf("%d\n",*p1);
    *p1=100; //此处会出错
```



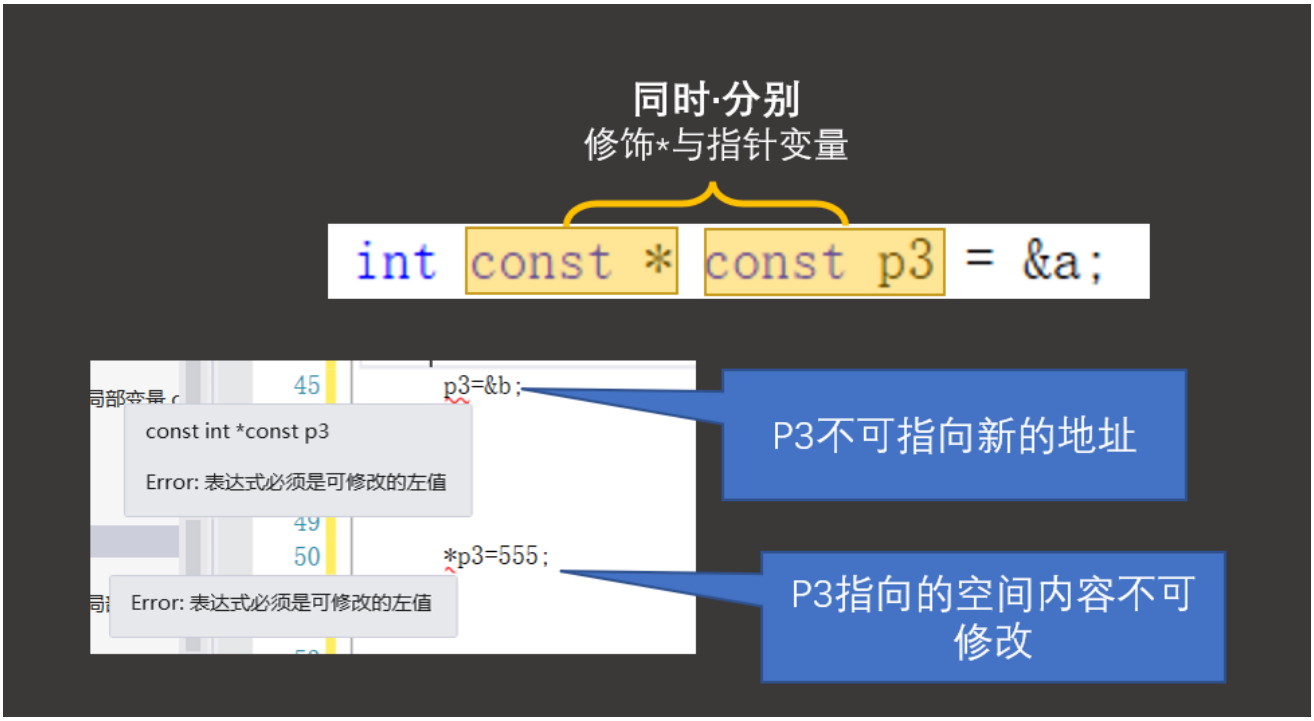
B：const 修饰指针变量 ——指向不可改 内容可改

```c
    p2=&b;//此处会出错
    *p2=111;
    printf("%d\n",*p2);
```



C：const 同时分别修饰*与指针变量 ——指向不可改 内容不可改

```
p3=&b;
*p3=555;
```

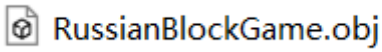


【结论：】const放到谁的前面，谁就不可改

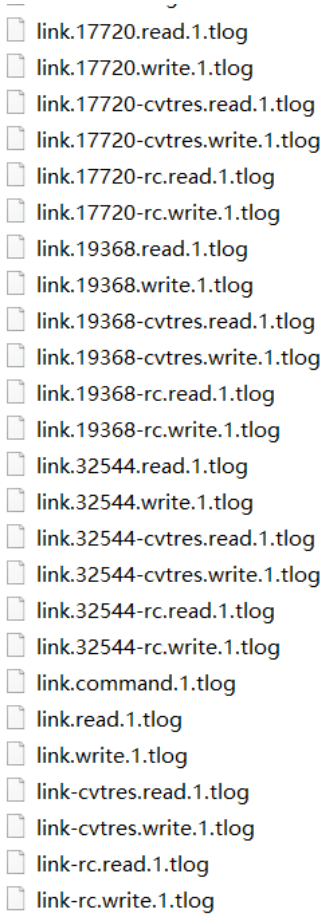
四：C 语言代码被运行的“心路历程”

①预处理期： 在这一阶段，源码中的所有**预处理语句**得到处理例如：**#include**语句所包含的文件内容**替换**掉语句本身，所有已定义的**宏**被展开。

②编译期：编译器对源码进行词法分析、语法分析、优化等操作，最后生成汇编代码。

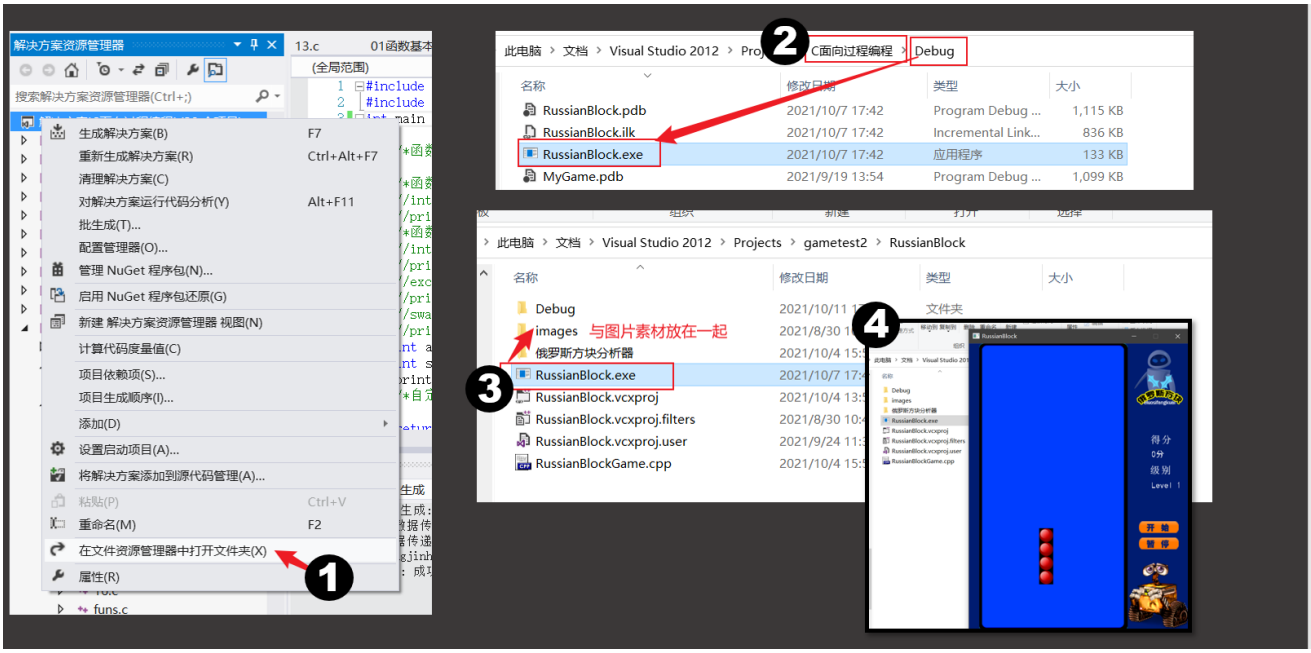
③汇编期：生成机器语言代码，保存在后缀为.obj的目标文件中 

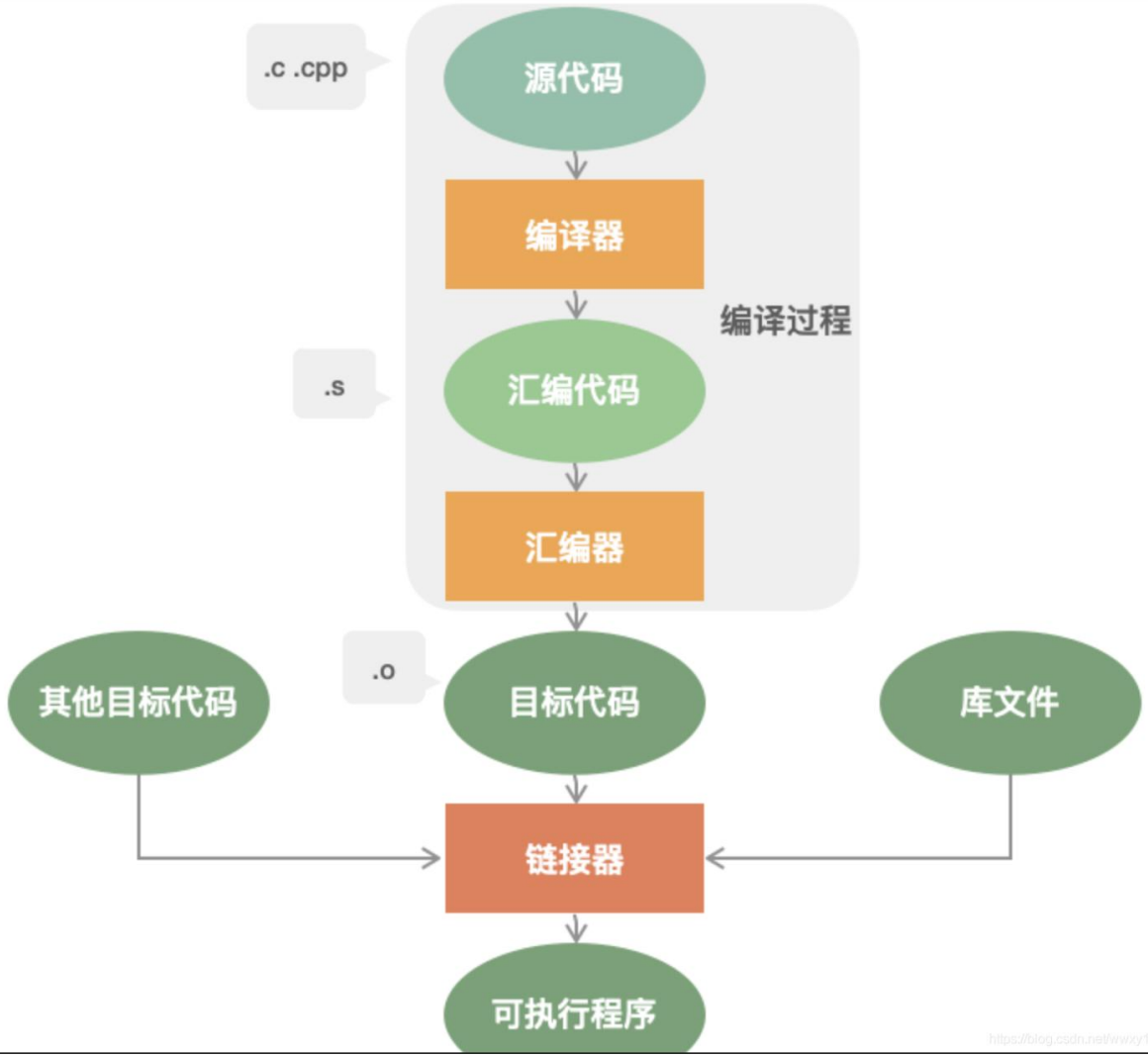
④链接期：经过汇编以后的机器代码还不能直接运行。为了使操作系统能够正确加载可执行文件，文件中必须包含固定格式的信息头，还必须与系统提供的启动代码链接起来才能正常运行，这些工作都是由链接器来完成的。



⑤执行期：执行.EXE文件，得到运行结果

如何找到exe文件并执行它：


解决方案资源管理器
搜索解决方案资源管理器(Ctrl+;)
生成解决方案(B) F7
重新生成解决方案(R) Ctrl+Alt+F7
清理解决方案(C)
对解决方案运行代码分析(Y) Alt+F11
批生成(T)...
配置管理器(O)...
管理 NuGet 程序包(N)...
启用 NuGet 程序包还原(G)
新建 解决方案资源管理器 视图(N)
计算代码度量值(C)
项目依赖项(S)...
项目生成顺序(I)...
添加(D)
设置启动项目(A)...
将解决方案添加到源代码管理(A)...
粘贴(P) Ctrl+V
重命名(M) F2
在文件资源管理器中打开文件夹(X)
属性(R)
1
2
此电脑 › 文档 › Visual Studio 2012 › Projects › C面向过程编程 › Debug
RussianBlock.pdb 2021/10/7 17:42 Program Debug ... 1,115 KB
RussianBlock.ilk 2021/10/7 17:42 Incremental Link... 836 KB
RussianBlock.exe 2021/10/7 17:42 应用程序 133 KB
MyGame.pdb 2021/9/19 13:54 Program Debug ... 1,099 KB
此电脑 › 文档 › Visual Studio 2012 › Projects › gametest2 › RussianBlock
Debug
images 与图片素材放在一起
俄罗斯方块分析器
RussianBlock.exe
RussianBlock.vcxproj
RussianBlock.vcxproj.filters
RussianBlock.vcxproj.user
RussianBlockGame.cpp
3
4


.c .cpp
源代码
编译器
编译过程
.s
汇编代码
汇编器
.o
其他目标代码
目标代码
库文件
链接器
可执行程序

五 #define 宏定义

①： 宏替换的基本用法

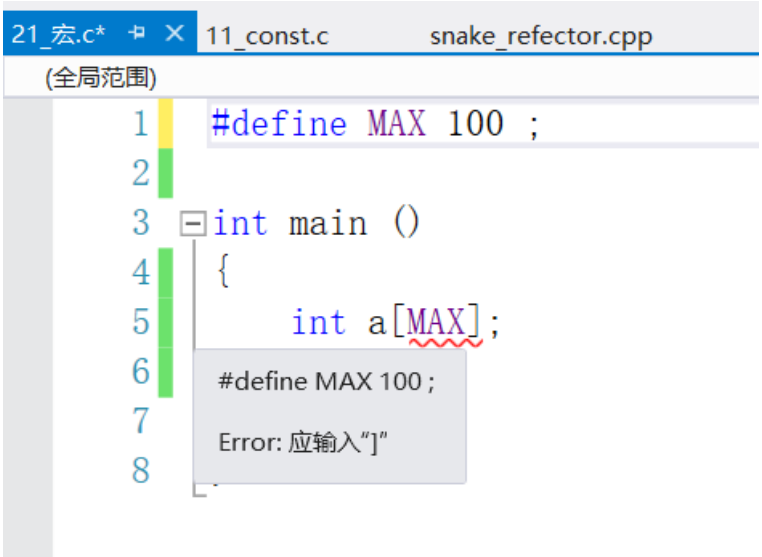
格式： **#define 宏替换名称 替换内容**

```
#define MAX 100
int main ()
{
    int a[MAX];
    return 0;
}
```

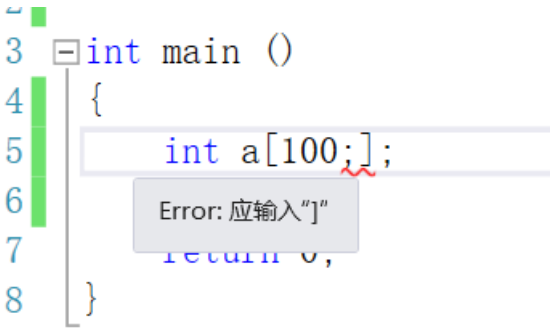
#开头为预编译的指令。也就是在编译之前C的运行系统会把所有的MAX的位置都“查找-替换 ”成 “100” 【注意：不要把100读作100，它就是三个字符一零零】因此你可以理解为它变身后的样子

```
int main ()
{
    int a[100];
    return 0;
}
```

【注意：】不要在 #define的语句末尾加;号，例如：



错误原因就是宏替换会把所有的MAX都进行文字替换即变成如下的样貌：



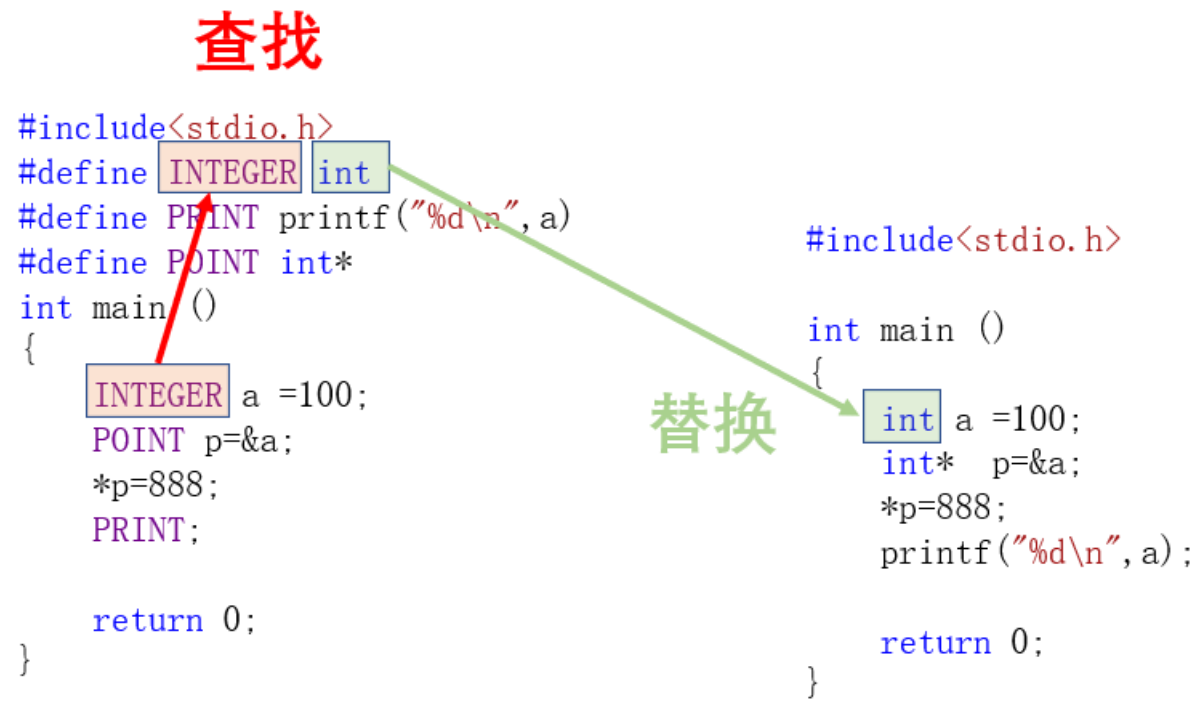
连错误提示都是一样的。【记住：】宏替换结尾不“随意乱加” ； 号。

②：用宏替换一个函数或表达式

```c
#include<stdio.h>
#define INTEGER int //宏 代替 类型
#define PRINT printf("%d\n",a) //宏 代替 函数
#define POINT int* // 宏 代替 指针类型
int main ()
{
    INTEGER a =100;//将替换成:  int a =100;
    POINT p=&a;//将替换成: int*  p=&a;
    *p=888;
    PRINT;//将替换成: printf("%d\n",a);

    return 0;
}
```

在预处理 宏替换 后 代码会变形如下：再然后进入编译过程。

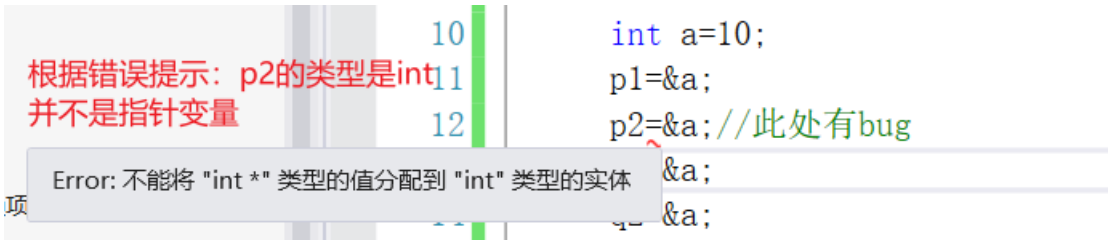
```c
#include<stdio.h>

int main ()
{
    int a =100;
    int*  p=&a;
    *p=888;
    printf("%d\n",a);
    return 0;
}
```



③：宏替换类型名 对比typedef 定义类型名的差异

```c
#include<stdio.h>

#define POINT_A int*
typedef int * POINT_B ;
int main ()
{
```

```
    POINT_A p1,p2;//预处理替换后： int* p1,p2; 即p1是指针变量 p2是普通int变量
    POINT_B q1,q2;//编译后：q1是指针变量   q2也是指针变量

    int a=10;
    p1=&a;
    p2=&a;//此处有bug
    q1=&a;
    q2=&a;

    return 0;
}
```



【差异点：】

1. #define执行的时点是预处理即编译前 typedef执行的时点是编译时
2. #define只是单纯的“查找替换”把源代码变成另外的样子，并没有创造新的类型名称。
3. typedef是定义的一个类型名，编译器把它当作一种新的类型来看待。

④：宏替换的嵌套

理解下面的代码：

```
#include<stdio.h>
#define A printf("helloworld.\n");
#define B A A A A A A A A A A
#define C B B B B B B B B B B
#define D C C C C C C C C C C
int main ()
{
    D
    return 0;
}
```

在没有循环的前提下打印出1000次的“helloworld.”

【总结：】宏的最重要的价值：增强可读性，与代码的灵活性。

⑤：用宏和typedef创建一个“布尔型数据”

```
#include<stdio.h>
typedef int BOOLEAN;//定义一个新类型名
#define True 1//将True全部查找替换成 1
#define False 0//将False全部查找替换成 0
int main ()
{
    BOOLEAN win=True;//对比： int win = 1 更加可读性
    return 0;
}
```

⑥：定义有参数的宏

```c
#include<stdio.h>
#define ADD(a,b) a * b
int main()
{
    printf("%d\n",ADD(3,4) );//替换成：printf("%d\n",3+4 )
    printf("%d\n",ADD(1+2,3+4) );//替换成：printf("%d\n",1+2+3+4 )
}
```

如果把上述代码 `#define ADD(a,b) a + b` 改成 `#define ADD(a,b) a * b` 那么打印结果是否会正确？？

⑦：定义一个可计算数组长度的宏

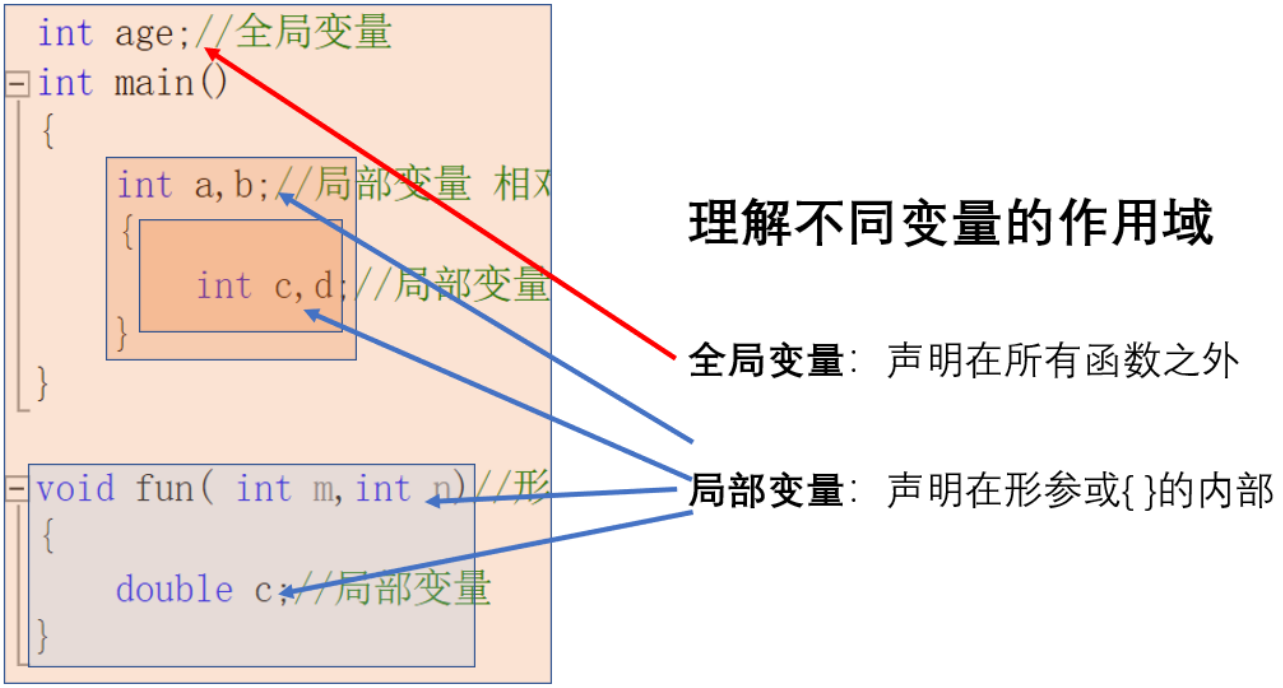
```c
#include<stdio.h>
#define LENGTH(arr_name)  sizeof(arr_name)/sizeof(arr_name[0])
int main()
{
    float arr[]={1.1,2.2,3.3,4.4,5.5};
    int i;
    for(i=0;i<LENGTH(arr) ;i++)//会被替换成：for(i=0;i<sizeof(arr)/sizeof(arr[0])  ;i++)
    {
        printf("%.1f\n",arr[i]);
    }
}
```

六、 变量作用域与extern 关键字

①：变量的作用域

所谓作用域就是变量可被访问的范围，可分为**全局变量**，即从声明位置开始，后面任何地方都可访问它。与之相反的，只能在一对{}内定义并在{}内可访问的变量称为**局部变量**。

```c
int age;//全局变量
int main()
{
    int a,b;//局部变量 相对于c、d，他们是c、d的全局变量
    {
        int c,d;//局部变量
    }
}

void fun( int m,int n)//形参也是局部变量
{
    double c;//局部变量
}
```



② 变量的生命周期

即变量何时创建 何时消失。

全局变量在程序启动时**创建**，程序完全关闭**消失**

局部变量在{}被执行时**创建**，{}执行结束后**消失**。{}可以是if switch for while do的作用范围，可以是函数体的边界；也可以是复合的多条语句构成的语句块。

③ extern外部变量的声明

所谓**外部变量**即在另外的源代码文件中定义的**全局变量**。如果当前的源代码文件要访问时；就需要用extern 声明一下，即告诉编译器，我目前用的变量是本代码文件以外定义的。用extern仅仅是声明告知一下。

【记住：声明≠定义】 当用extern 关键词声明外部变量时：如果给变量赋初始值就变成了变量的定义：即在内存中创建一个新的变量的意思，这就产生了变量重复定义的编译错误。

```
//a.c  文件
int age = 1;//全局变量默认值为 0
```

```
//b.c 文件
int age=100;
int main()
{
    printf("%d\n",age);
    return 0;
}
```

上面代码：两个int型同名全局变量age被定义赋予初始值。将会报出如下错误提示：`: _age 已经在 a.obj 中定义 fatal error LNK1169: 找到一个或多个多重定义的符号`

也就是说：变量同名重复定义了。

建议如下写法：

```c
#include<stdio.h>
extern int age ;//只声明——：告知用到外部的一个全局变量；不定义——：不赋初始值就不会创建存储空间
int main()
{
    age=100;//在需要的时候赋值外部变量
    printf("%d\n",age);
    return 0;
}
```

假如一个项目里的【仅有两个文件】a.c与b.c的代码如下：

```c
//a.c源代码
#include<stdio.h>
extern int age ;
void fun()
{
    printf("fun 里打印 %d\n",age);
}
```

```c
//b.c源代码
#include<stdio.h>
extern int age ;
void fun();
int main()
{
    ////在需要的时候赋值外部变量
   fun();
   age=100;
   fun();
   return 0;
}
```

两个文件都有extern int age ; 也就是都声明此变量为外部变量，这个时候编译器就会在本项目内寻找age变量的定义在哪里，如果只都是声明而没有一处定义则报出如下错误提示：

```
1>  正在生成代码...
1>a.obj : error LNK2001: 无法解析的外部符号 _age
1>b.obj : error LNK2001: 无法解析的外部符号 _age
fatal error LNK1120: 1 个无法解析的外部命令
```

也就是找不到 a.c 与 b.c声明的外部变量。 因此在一个项目内，一个全局变量尽量做到【仅在一处定义，可以多处声明】

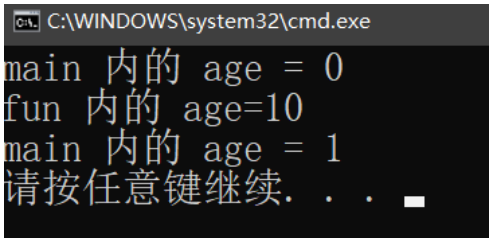
当然，如果把a.c修改如下：

```c
//a.c源代码
#include<stdio.h>
extern int age = 1;//当赋值后就起到了定义变量创建空间的作用，如果没有全局变量重名程序可正常执行。
                   //此处extern关键字无效
void fun()
{
    printf("fun 里打印 %d\n",age);
}
```

④ 全局变量与局部变量重名

在一个函数内部全局变量与局部变量重名时，同名的全局变量在函数内部不起作用。

```
#include <stdio.h>
int age  ;
void fun()
{
    int age=10;//定义的 局部变量与全局同名
    printf("fun 内的 age=%d\n",age);//此刻访问的是局部变量age
    age++;//此刻访问的是局部变量age
}
int main()
{
    printf("main 内的 age = %d\n",age);
    age++;
    fun();
    printf("main 内的 age = %d\n",age);
    return 0;
}
```



七、 static静态变量

static称之为：静态，表示唯一的概念。

1. static修饰的局部变量 是唯一存在的，它被创建于全局变量区，但不会被全局访问到，只会被其所在函数体内访问。
2. 它不会随着函数执行的结束而消失。当函数又一次被执行时，还会继续访问它最后一次的赋值

```
#include <stdio.h>

void fun41()
{
    /*static 理解为局部内的 唯一变量 相当于全局，但全局不可访问  */
    static int c;//static 修饰的变量会放在全局变量区，默认值为0;
    int i;
    i=10;
    i++;
    c++;
    printf("c=%d,i=%d\n",c,i);
}
int main  ()
{
    fun41();
    fun41();
    fun41();
    return 0;
}
```

```
C:\WINDOWS\system32\cmd.exe
c=1, i=11
c=2, i=11
c=3, i=11
请按任意键继续. . .
```

通过结果可知：int i为普通局部变量，函数fun14每一次被调用，它都会重新创建，函数执行完它即消失。static int c变量则完全相反：只被创建一次，函数执行完不会消失。

01枚举enum.c　fun.cpp　RussianBlockGame.cpp　mylib.h　31_extern与全局

(全局范围)

```c
#include <stdio.h>

void fun41()
{
    /*static 理解为局部内的 唯一变量 相当于全局，但全局不
    static int c;//static 修饰的变量会放在全局变量区，默
    int i;
    i=10;
    i++;
    c++;
    printf("c=%d, i=%d\n", c, i);
}
int main ()
{
    fun41();
    fun41();
    fun41();
    printf("c = %d", c);
}
```

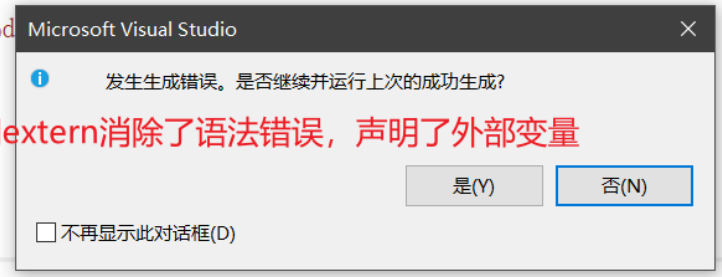
虽然c变量放在全局区

但是它不会被全局访问

Error: 未定义标识符 "c"

```
    /*static 理解为局部内的 唯一变量 相当于全局，但全局不可访问  */
    static int c;//static 修饰的变量会放在全局变量区，默认值为0;
    int i;
    i=10;
    i++;
    c++;
    printf("c=%d, i=%d
}
int main  ()
{
    extern int c;
    fun41();
    fun41();
    fun41();
    printf("c = %d",c);
    return 0;
```

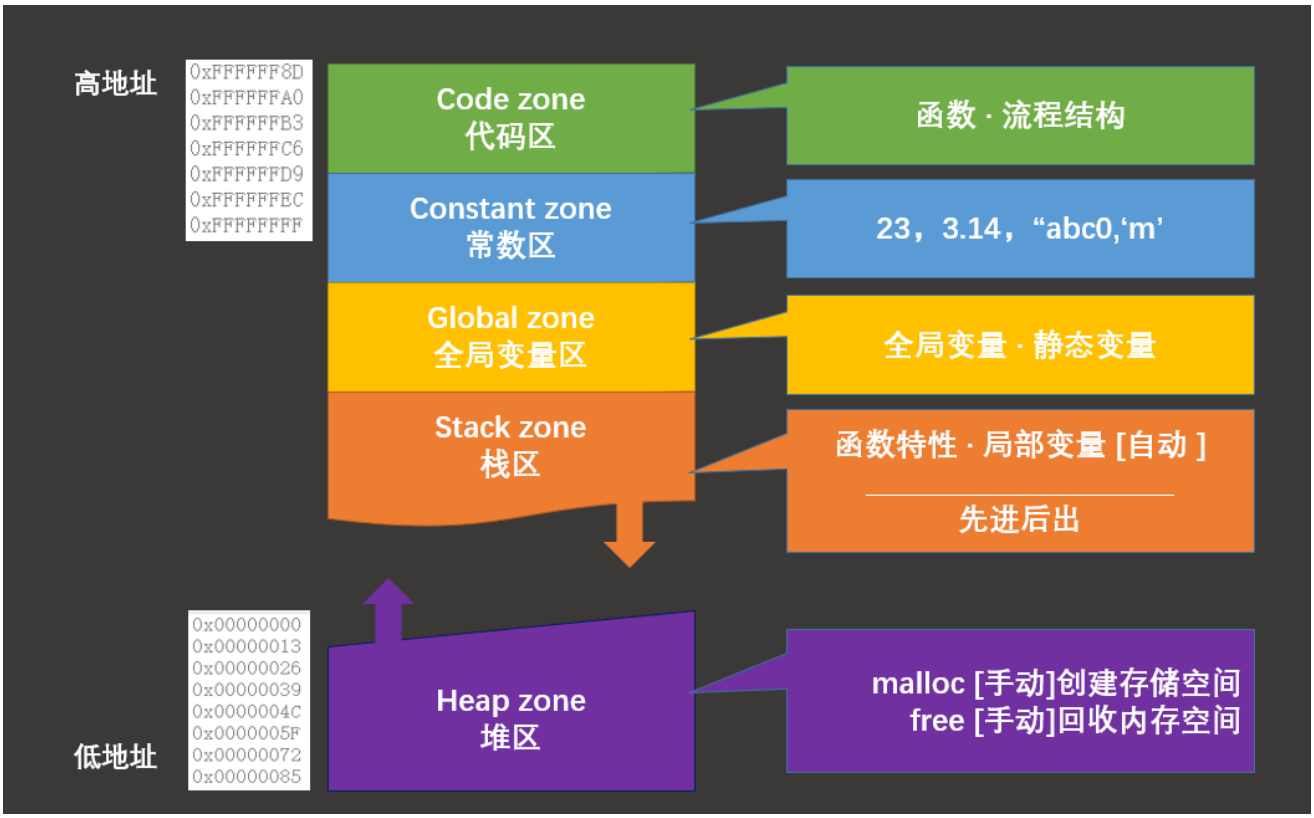
即使用extern消除了语法错误，声明了外部变量




```
来源(S): 生成
-- 已启动生成: 项目: C_3_2, 配置: Debug Win32 ------
static静态变量.c
atic静态变量.obj : error LNK2001: 无法解析的外部符号 _c
ers\zhangjinhai\Documents\Visual Studio 2012\Projects\gametest2\Debug\C_3_2.exe : fatal error LNK1120: 1 个无法解析的外部命令
==== 生成: 成功 0 个，失败 1 个，最新 0 个，跳过 0 个 ==========
```

当扔无法解析访问

八、 C语言的内存区域



阅读下列代码，理解不同语素所处的区域。

```
#include <stdio.h>
int sum(int a )//sum 代码区  a 栈区
{
    static int c;//c 全局变量区
```

```
    c+=a;
    return c;
}
int age ;//age 全局变量区
int main ()
{
    printf("%d\n",sum(10));// 10 常数区
    printf("%d\n",sum(5));// 5 常数区
    printf("%d\n",sum(1));// 1 常数区
    return 0;// 0 常数区
}
```

堆区：可自定义的更大存储空间，但需要手动撤销回收空间（属于租借模式，用完记得还）。

```
#include <stdlib.h>
int main ()
{
    /*堆区*/
    int *p = (int *)malloc(4);//申请4个字节的空间 也可用 sizeof(int) 得到某类型的字节数
    *p=200;//利用 *p可以对堆空间进行数据存取。
    printf("%d\n",*p);
    free(p);/*释放p指向的空间*/
    printf("%d\n",*p);//p指向的空间内容将不确定
    return 0;
}
```

序号	函数和描述
1	**void *calloc(int num, int size);** 在内存中动态地分配 num 个长度为 size 的连续空间，并将每一个字节都初始化为 0。所以它的结果是分配了 num*size 个字节长度的内存空间，并且每个字节的值都是0。
2	**void free(void *address);** 该函数释放 address 所指向的内存块,释放的是动态分配的内存空间。
3	**void *malloc(int num);** 在堆区分配一块指定大小的内存空间，用来存放数据。这块内存空间在函数执行完成后不会被初始化，它们的值是未知的。
4	**void *realloc(void *address, int newsize);** 该函数重新分配内存，把内存扩展到 **newsize**。